

NEXUS HRMS PLATFORM

Security Features Documentation

Comprehensive Security Architecture & Compliance
Reference

Version 1.0

SaaS-Based AI HRMS Platform

June 2026

Table of Contents

1. Security Architecture Overview	3
1.1 Security Stack	3
1.2 Core Security Parameters	3
2. Authentication System	4
2.1 Login Flow	4
2.2 Token Creation Details	4
2.3 Token Verification	5
2.4 Logout Process	5
2.5 Password Hashing	6
3. Authorization & RBAC	6
3.1 Database Roles (5 Roles)	6
3.2 Application Roles (14 Roles)	6
3.3 Sidebar Role Filtering	7
3.4 API Route Protection	8
4. Data Security	8
4.1 Multi-Tenant Data Isolation	8
4.2 Database Security	9
4.3 Prisma ORM Security Benefits	9
5. API Security	10
5.1 Bearer Token Authentication	10
5.2 CORS Configuration	10
5.3 Rate Limiting Considerations	10
5.4 Input Validation	11
6. Infrastructure Security	11
6.1 Vercel Deployment Security	11
6.2 Neon PostgreSQL Security	12
6.3 Environment Variable Management	12

7. Audit & Compliance	13
7.1 AuditLog Model	13
7.2 LoginActivity Tracking	13
7.3 Notification System for Security Events	14
8. Vulnerability Assessment	14
8.1 Critical Vulnerabilities	14
8.2 High Severity Issues	15
8.3 Medium Severity Issues	15
9. Security Recommendations	16
9.1 Critical Priority Recommendations	16
9.2 High Priority Recommendations	16
9.3 Medium Priority Recommendations	17
9.4 Standard Priority Recommendations	17
10. Data Privacy & GDPR Considerations	18
10.1 GDPR Compliance Considerations	18
10.2 Data Retention & Deletion Policy	19
10.3 Privacy by Design Principles	20

1. Security Architecture Overview

The NEXUS HRMS platform implements a multi-layered security architecture designed to protect sensitive human resources data across the entire application stack. The security model encompasses authentication, authorization, data protection, API security, infrastructure hardening, and continuous monitoring.

1.1 Security Stack

The NEXUS security stack operates across five distinct layers, each providing complementary protections:

- **Presentation Layer:** Next.js client with role-based UI filtering and secure token storage in localStorage with Bearer + cookie token retrieval mechanism.
- **Application Layer:** Next.js API routes with JWT-based authentication middleware, input validation, and CORS enforcement.
- **Authentication Layer:** JWT tokens signed with HS256 algorithm using the jose library, with bcryptjs password hashing at 12 salt rounds and 24-hour token expiry.
- **Data Layer:** Prisma ORM with multi-tenant isolation via tenantId foreign keys, parameterized queries preventing SQL injection, and Neon serverless PostgreSQL with encrypted connections.
- **Infrastructure Layer:** Vercel deployment with automatic TLS, Neon PostgreSQL with encryption at rest and in transit, and environment variable management through Vercel secrets.

1.2 Core Security Parameters

Parameter	Implementation	Value/Detail
Authentication Method	JWT (JSON Web Tokens)	HS256 algorithm via jose library
Password Hashing	bcryptjs	12 salt rounds (cost factor 12)
Token Expiry	JWT expiration	24 hours (86400 seconds)
Token Storage	Client-side	localStorage with Bearer + cookie retrieval
Database Isolation	Multi-tenant	tenantId foreign key on all tenant-scoped models
SQL Injection Prevention	Prisma ORM	Parameterized queries, no raw SQL
Connection Encryption	Neon PostgreSQL	TLS/SSL encrypted connections

Deployment Security	Vercel	Automatic HTTPS, edge network
CORS Policy	Next.js middleware	Configured origin validation
Secret Management	Vercel Environment Variables	Encrypted at rest, never exposed to client

2. Authentication System

The NEXUS authentication system implements JWT-based stateless authentication with secure password handling and comprehensive session management. The system uses the jose library for JWT operations and bcryptjs for password hashing.

2.1 Login Flow

The authentication flow follows a standard credential-based login process with JWT token issuance:

- **Step 1 — Credential Submission:** User submits email and password through the login form. The client sends a POST request to `/api/auth/login`.
- **Step 2 — User Lookup:** The API route queries the database using Prisma to find the user by email. The query includes the user record with associated role and tenant information.
- **Step 3 — Password Verification:** The submitted password is compared against the stored bcrypt hash using `bcryptjs.compare()`. The bcrypt algorithm with 12 salt rounds ensures computationally expensive brute-force resistance.
- **Step 4 — Token Creation:** Upon successful verification, a JWT token is created using `jose.SignJWT()`. The payload includes `userId`, `email`, `role`, and `tenantId` claims. The token is signed with HS256 using the `JWT_SECRET` from environment variables.
- **Step 5 — Token Delivery:** The JWT token is returned in the response body. The client stores the token in `localStorage` for subsequent API calls.
- **Step 6 — Login Activity Logging:** The system creates a `LoginActivity` record capturing the login timestamp, IP address (if available), and user agent for audit purposes.

2.2 Token Creation Details

JWT tokens in NEXUS are created with the following structure and security parameters:

Property	Value	Notes
Algorithm	HS256	HMAC with SHA-256, symmetric key
Signing Library	jose (SignJWT)	Standards-compliant JWT library
Expiration	24 hours	Configurable via environment variable
Payload Claims	userId, email, role, tenantId	Minimal necessary claims
Secret Key	JWT_SECRET env variable	Must be 256-bit minimum for HS256
Token Format	JWS Compact Serialization	Standard three-part JWT format

2.3 Token Verification

Every protected API route verifies the JWT token using the jose library's `jwtVerify()` function. The verification process extracts the token from the Authorization header (Bearer token) and validates the signature, expiration, and claims.

- Extract the Authorization header from the incoming request.
- Parse the Bearer token from the header value.
- Call `jose.jwtVerify()` with the token and `JWT_SECRET`.
- Validate that the token has not expired and the signature is valid.
- Extract `userId`, `role`, and `tenantId` from the verified payload.
- Attach the decoded user context to the request for downstream processing.
- If verification fails, return 401 Unauthorized response.

2.4 Logout Process

The logout process clears the JWT token from client-side storage. Since NEXUS uses stateless JWT authentication, server-side token invalidation is not natively supported. The logout flow:

- Client calls the logout endpoint or clears local token.
- Token is removed from `localStorage`.
- Application state is reset to unauthenticated.
- User is redirected to the login page.
- `LoginActivity` record is optionally updated with logout timestamp.

2.5 Password Hashing

Password security is implemented using bcryptjs with the following parameters:

Parameter	Value	Security Rationale
Algorithm	bcrypt	Adaptive hashing, resistant to GPU/ASIC attacks
Salt Rounds	12	Cost factor of $2^{12} = 4096$ iterations
Hash Length	60 characters	Standard bcrypt output format
Password Validation	bcryptjs.compare()	Constant-time comparison prevents timing attacks
Password Storage	Hash only	Original password never stored or logged

3. Authorization & RBAC

NEXUS implements a Role-Based Access Control (RBAC) system with dual role models: database-level roles for authentication and application-level roles for fine-grained authorization. This section details the role hierarchy, permission mapping, and access control mechanisms.

3.1 Database Roles (5 Roles)

The database User model defines 5 core roles that determine the user's position in the organizational hierarchy:

Role	Scope	Key Capabilities
SUPER_ADMIN	Platform-wide	Full system access, tenant management, all operations
ADMIN	Organization-wide	Organization management, user administration, all modules
HR	Organization-wide	HR module access, employee management, payroll, recruitment
MANAGER	Department-level	Team management, approvals, performance reviews
EMPLOYEE	Individual-level	Self-service, leave requests, profile updates

3.2 Application Roles (14 Roles)

The application sidebar and route protection use an expanded set of 14 roles for more granular access control:

App Role	DB Role Mapping	Sidebar Access
super_admin	SUPER_ADMIN	All modules + Tenant Management + AI Admin
admin	ADMIN	All modules within tenant organization
hr	HR	Employee, Recruitment, Onboarding, Leave, Payroll, Performance
manager	MANAGER	Team view, Approvals, Performance, Attendance, Leave
employee	EMPLOYEE	Self-service, Leave, Attendance, Helpdesk, Training
recruiter	HR	Recruitment, AI Interview, Offers, Candidate Management
interviewer	HR / MANAGER	AI Interview, Candidate Assessment
payroll_admin	HR	Payroll, Tax, Reports, Salary Structure
trainer	HR / MANAGER	Training, Course Management, Assessments
finance	HR / ADMIN	Payroll, Expenses, FNF, Loans, Travel
it_admin	ADMIN	Asset Management, IT Provisioning, Helpdesk
helpdesk_agent	EMPLOYEE / HR	Helpdesk, AI Assistant, Ticket Management
compliance_officer	HR / ADMIN	Compliance, Audit Logs, Reports
viewer	EMPLOYEE	Read-only access to permitted modules

3.3 Sidebar Role Filtering

The NEXUS sidebar navigation implements role-based filtering to ensure users only see menu items relevant to their assigned role. The filtering mechanism works as follows:

- Upon login, the user's role is extracted from the JWT token payload.
- The sidebar configuration maps each menu item to an array of permitted roles.
- The rendering engine filters menu items based on the current user's role.
- Only authorized menu items are displayed in the sidebar navigation.
- Direct URL access to unauthorized routes is handled by client-side route guards.

- API-level protection ensures that even if UI is bypassed, data access is controlled.

3.4 API Route Protection

API route protection in NEXUS is implemented through authentication middleware that verifies the JWT token on protected routes. The protection operates at multiple levels:

- **Token Verification:** All protected API routes extract and verify the JWT token from the Authorization header using `jose.jwtVerify()`.
- **Role-Based Access:** Certain API routes check the user's role from the token payload to determine if the requested operation is permitted.
- **Tenant Isolation:** API routes that query tenant-scoped data include `tenantId` filtering from the token payload to ensure data isolation.
- **Ownership Validation:** Employee-level operations validate that the authenticated user is accessing only their own data (e.g., own leave requests, own profile).
- **Important Note:** The current middleware implementation includes a pass-through for unauthenticated routes, which means some routes may not enforce authentication consistently. See Section 7 for details.

4. Data Security

Data security in NEXUS HRMS is critical given the sensitivity of HR data including personal information, salary details, performance records, and medical information. The platform implements multiple layers of data protection.

4.1 Multi-Tenant Data Isolation

NEXUS implements multi-tenant data isolation at the database level using `tenantId` foreign keys on all tenant-scoped models. This approach ensures that each organization's data is logically separated within the shared database infrastructure.

- **tenantId Foreign Key:** Every tenant-scoped model (Employee, Leave, Payroll, etc.) includes a `tenantId` field that references the Tenant model. All queries include `tenantId` filtering to ensure data isolation.
- **Shared Database, Logical Isolation:** Unlike separate database per tenant, NEXUS uses a single database with logical isolation through `tenantId`. This reduces infrastructure overhead while maintaining data separation.

- **Query-Level Enforcement:** Prisma queries must explicitly include `tenantId` in where clauses. The system relies on API route implementations to consistently include `tenantId` filtering.
- **Cross-Tenant Access Prevention:** The JWT token includes the `tenantId` claim, and API routes use this to filter queries. This prevents a user from one tenant accessing another tenant's data through the API.
- **Super Admin Scope:** The `SUPER_ADMIN` role has access to all tenant data for platform management. This is an intentional design decision for administrative operations.

4.2 Database Security

The NEXUS database layer leverages Neon serverless PostgreSQL with built-in security features:

Security Feature	Implementation	Protection Level
Connection Encryption	Neon SSL/TLS for all connections	Data in transit encryption
SQL Injection Prevention	Prisma ORM parameterized queries	No raw SQL execution
Access Control	Database credentials via env vars	Credential isolation
Connection Pooling	Neon serverless connection pooling	Connection limit protection
Database Hosting	Neon cloud (AWS infrastructure)	Physical security by provider
Backup & Recovery	Neon point-in-time recovery	Data durability
Schema Migration	Prisma migrate with versioning	Controlled schema changes

4.3 Prisma ORM Security Benefits

The use of Prisma ORM provides several security advantages for data access:

- **SQL Injection Prevention:** Prisma generates parameterized queries automatically, eliminating the risk of SQL injection attacks through user input.
- **Type Safety:** Prisma's generated client provides TypeScript type safety, reducing runtime errors and potential data access vulnerabilities.
- **Query Validation:** Prisma validates query structure before execution, preventing malformed queries from reaching the database.
- **Schema Enforcement:** The Prisma schema defines data types, relations, and constraints, ensuring data integrity at the application level.
- **Migration Safety:** Prisma Migrate provides controlled schema changes with rollback capability, reducing the risk of data corruption during updates.

5. API Security

API security encompasses the protection of all HTTP endpoints exposed by the NEXUS platform. This section covers authentication mechanisms, CORS configuration, rate limiting considerations, and input validation practices.

5.1 Bearer Token Authentication

All protected API endpoints require a valid JWT Bearer token in the Authorization header. The authentication flow for API requests:

- Client retrieves the JWT token from localStorage.
- Token is included in the Authorization header as "Bearer <token>".
- API middleware extracts the token and verifies it using jose.jwtVerify().
- If verification succeeds, the request proceeds with the decoded user context.
- If verification fails, the API returns a 401 Unauthorized response.
- Expired tokens require the user to re-authenticate through the login flow.

5.2 CORS Configuration

Cross-Origin Resource Sharing (CORS) is configured through Next.js middleware to control which domains can access the API endpoints:

- **Allowed Origins:** Only the deployed application domain is permitted for API access in production.
- **HTTP Methods:** GET, POST, PUT, PATCH, DELETE are allowed as needed for API operations.
- **Headers:** Authorization, Content-Type, and custom headers are permitted for cross-origin requests.
- **Credentials:** Cookie-based credentials are supported for specific authentication flows.
- **Preflight Handling:** OPTIONS requests are handled automatically by the Next.js middleware for CORS preflight checks.

5.3 Rate Limiting Considerations

Rate limiting is an important consideration for protecting the API from abuse, brute-force attacks, and denial-of-service attempts:

- **Current Status:** Application-level rate limiting is not currently implemented in the base NEXUS deployment. Reliance is placed on Vercel's platform-level rate limiting.
- **Authentication Endpoints:** The login endpoint (/api/auth/login) is particularly vulnerable to brute-force attacks and should be rate-limited.
- **AI Endpoints:** AI-powered endpoints (/api/ai-interview, /api/ai-chat) should have rate limiting to control API costs.
- **Recommended Approach:** Implement rate limiting using Vercel KV with Upstash for distributed rate limiting across edge functions.
- **Rate Limit Thresholds:** Suggested limits: Login - 5 attempts per minute; API - 100 requests per minute; AI endpoints - 20 requests per minute per user.

5.4 Input Validation

Input validation is performed at both the client and server levels to prevent injection attacks and data corruption:

- **Client-Side Validation:** Form validation using React Hook Form with Zod schema validation provides immediate feedback and prevents obviously invalid data submission.
- **Server-Side Validation:** API routes should validate all input data before processing. Current implementation varies across routes; consistent validation is recommended.
- **Data Sanitization:** All user inputs should be sanitized before database storage to prevent XSS (Cross-Site Scripting) and injection attacks.
- **File Upload Validation:** Document and image uploads should validate file type, size, and content before processing.
- **Type Checking:** TypeScript provides compile-time type checking, reducing runtime type-related vulnerabilities.

6. Infrastructure Security

Infrastructure security encompasses the hosting platform, database infrastructure, and deployment pipeline that underpin the NEXUS HRMS platform.

6.1 Vercel Deployment Security

Feature	Implementation	Security Benefit
---------	----------------	------------------

HTTPS/TLS	Automatic SSL certificates	Encrypted data in transit
DDoS Protection	Vercel edge network	Distributed attack mitigation
Deployment Isolation	Serverless function isolation	Process-level separation
Access Control	Vercel team authentication	Authorized deployments only
Audit Logging	Deployment activity logs	Change tracking
Edge Network	Global CDN distribution	Performance + availability
Preview Deployments	Branch-based previews	Safe testing environment

6.2 Neon PostgreSQL Security

- **Encryption at Rest:** Neon encrypts all data at rest using AES-256 encryption on the underlying storage infrastructure.
- **Encryption in Transit:** All connections to Neon PostgreSQL use SSL/TLS encryption, ensuring data security during transmission.
- **Network Isolation:** Neon databases are hosted in isolated VPC environments within AWS infrastructure.
- **Access Control:** Database access is controlled through connection strings with credentials stored as Vercel environment variables.
- **Point-in-Time Recovery:** Neon provides point-in-time recovery capabilities for data restoration in case of accidental deletion or corruption.
- **Branching:** Neon's database branching feature allows creating isolated database copies for testing without affecting production data.
- **Auto-Suspend:** Neon serverless computes auto-suspend after inactivity, reducing the attack surface during idle periods.

6.3 Environment Variable Management

Environment variables in NEXUS are managed through Vercel's encrypted environment variable system:

- **JWT_SECRET:** The secret key used for signing and verifying JWT tokens. Must be a strong, random string of at least 32 characters.
- **DATABASE_URL:** The Neon PostgreSQL connection string. Contains host, credentials, and SSL parameters.
- **NEXT_PUBLIC_* Variables:** Variables prefixed with NEXT_PUBLIC_ are exposed to the client bundle. Only non-sensitive configuration should use this prefix.

- **Encryption:** All environment variables are encrypted at rest in Vercel's infrastructure.
- **Environment Scoping:** Variables can be scoped to Production, Preview, or Development environments.
- **Access Control:** Only authorized team members with Vercel project access can view or modify environment variables.
- **Warning:** The current codebase includes a fallback JWT secret (your-secret-key) which is a critical security vulnerability. See Section 7.

7. Audit & Compliance

The NEXUS platform includes audit logging and compliance tracking capabilities to maintain accountability, support incident investigation, and meet regulatory requirements.

7.1 AuditLog Model

The AuditLog model captures a comprehensive record of significant actions across the platform:

Field	Type	Purpose
id	Int (Auto)	Unique audit log entry identifier
action	String	Description of the action performed
entity	String	Target entity type (e.g., Employee, Leave)
entityId	String	Identifier of the affected record
userId	String	ID of the user who performed the action
tenantId	String	Tenant context for multi-tenant isolation
details	String (Optional)	Additional context or change details
ipAddress	String (Optional)	IP address of the client
timestamp	DateTime	When the action occurred (auto-set)

7.2 LoginActivity Tracking

The LoginActivity model specifically tracks user authentication events for security monitoring:

- **Login Events:** Every successful login is recorded with timestamp, user ID, and session details.

- **Failed Login Attempts:** Failed authentication attempts should be logged to detect brute-force attacks (implementation recommended).
- **Session Duration:** Login and logout timestamps enable calculation of session duration for compliance reporting.
- **Geographic Analysis:** IP address tracking (when available) enables geographic analysis of login patterns for anomaly detection.
- **Multi-Device Detection:** Concurrent session tracking can identify unusual access patterns indicating potential account compromise.

7.3 Notification System for Security Events

The NEXUS notification system provides real-time alerts for security-relevant events:

- **Login Notifications:** Users receive notifications for login events on their account.
- **Role Change Alerts:** Changes to user roles trigger notifications to both the affected user and administrators.
- **Data Access Alerts:** Sensitive data access (salary information, personal records) can be configured to generate audit notifications.
- **Bulk Operation Warnings:** Bulk data export or modification operations trigger admin notifications.
- **System Alerts:** Platform-level security events (failed login spikes, unusual API patterns) alert the Super Admin.

8. Vulnerability Assessment

This section documents known security gaps and vulnerabilities identified in the current NEXUS HRMS implementation. These items require attention for production deployment and are prioritized by severity.

8.1 Critical Vulnerabilities

Vulnerability	Severity	Description	Impact
JWT Fallback Secret	Critical	Code includes fallback secret "your-secret-key" when JWT_SECRET env var is not set	Attackers can forge valid JWT tokens with known secret

No Server-Side Route Protection	Critical	Pages are accessible via direct URL without server-side auth check	Unauthorized data access bypassing client-side guards
Middleware Pass-Through	Critical	Middleware allows unauthenticated requests through to protected routes	API endpoints accessible without valid tokens

8.2 High Severity Issues

Vulnerability	Severity	Description	Impact
Role Mismatch	High	5 DB roles vs 14 app roles create authorization inconsistencies	Users may access unauthorized features
No Rate Limiting	High	API endpoints lack application-level rate limiting	Brute-force and DoS attack vulnerability
Token in localStorage	High	JWT stored in localStorage is vulnerable to XSS	Token theft through XSS attacks
No Refresh Tokens	High	Single 24h token with no refresh mechanism	Forced re-authentication or extended vulnerability window
No CSRF Protection	High	No CSRF tokens for state-changing operations	Cross-site request forgery attacks

8.3 Medium Severity Issues

Vulnerability	Severity	Description	Impact
Inconsistent Input Validation	Medium	Input validation varies across API routes	Potential injection attacks on weak routes
No Password Policy	Medium	No minimum password complexity requirements	Weak passwords susceptible to cracking
No Account Lockout	Medium	No lockout after failed login attempts	Unlimited brute-force attempts possible
Error Information Leakage	Medium	Some API errors expose internal details	Information disclosure to attackers

No Security Headers	Medium	Missing CSP, X-Frame-Options, etc.	Clickjacking and XSS vulnerability
Incomplete Audit Logging	Medium	Not all critical operations are logged	Gaps in forensic investigation capability

9. Security Recommendations

Based on the vulnerability assessment and security architecture review, the following recommendations are provided in priority order for strengthening the NEXUS HRMS security posture.

9.1 Critical Priority Recommendations

- **R1 — Remove JWT Fallback Secret:** Remove the fallback "your-secret-key" from the JWT signing code. The application should fail to start if JWT_SECRET is not configured, not silently use an insecure default.
- **R2 — Implement Server-Side Route Protection:** Add server-side authentication checks to all page routes using Next.js middleware or getServerSideProps. Never rely solely on client-side route guards.
- **R3 — Fix Middleware Pass-Through:** Update the middleware to block unauthenticated requests to protected routes. Remove or restrict the pass-through logic that allows requests without valid tokens.
- **R4 — Implement Rate Limiting:** Add rate limiting to all API endpoints, especially authentication endpoints. Use Vercel KV with Upstash for distributed rate limiting.
- **R5 — Move Token Storage to HttpOnly Cookies:** Replace localStorage token storage with HttpOnly, Secure, SameSite cookies to protect against XSS-based token theft.

9.2 High Priority Recommendations

- **R6 — Implement Refresh Token Rotation:** Add short-lived access tokens (15-30 minutes) with longer-lived refresh tokens that rotate on each use. This limits the damage window of token compromise.
- **R7 — Unify Role Model:** Reconcile the 5 DB roles and 14 app roles into a consistent authorization model. Implement a role-permission mapping table that both DB and UI reference.

- **R8 — Add CSRF Protection:** Implement CSRF tokens for all state-changing operations (POST, PUT, DELETE). Use the double-submit cookie pattern or synchronizer token pattern.
- **R9 — Implement Password Policy:** Enforce minimum password complexity (8+ characters, mixed case, numbers, special characters). Add password strength meter on the registration form.
- **R10 — Add Account Lockout:** Implement progressive lockout after failed login attempts (e.g., lock after 5 failures, unlock after 15 minutes or admin action).

9.3 Medium Priority Recommendations

- **R11 — Standardize Input Validation:** Create a shared validation middleware using Zod schemas for all API routes. Ensure consistent validation across all endpoints.
- **R12 — Add Security Headers:** Implement Content-Security-Policy, X-Frame-Options, X-Content-Type-Options, and Referrer-Policy headers using Next.js configuration.
- **R13 — Implement Comprehensive Audit Logging:** Ensure all CRUD operations on sensitive models (Employee, Payroll, User) are logged with full change details including before/after values.
- **R14 — Add File Upload Security:** Validate file type (magic bytes, not just extension), enforce size limits, scan for malware, and store uploads outside the web root.
- **R15 — Implement Data Encryption at Application Level:** Encrypt particularly sensitive fields (SSN, bank account numbers) at the application level in addition to database-level encryption.
- **R16 — Add Session Management:** Implement server-side session tracking with the ability to invalidate specific sessions, force logout, and detect concurrent sessions.
- **R17 — Implement API Versioning:** Add API versioning to enable controlled security updates without breaking existing integrations.
- **R18 — Add Dependency Scanning:** Integrate automated dependency vulnerability scanning (npm audit, Snyk) into the CI/CD pipeline.

9.4 Standard Priority Recommendations

- **R19 — Implement Content Security Policy:** Define a strict CSP that prevents inline script execution and limits resource loading to trusted sources.

- **R20 — Add Subresource Integrity:** Add SRI hashes to all externally loaded scripts and stylesheets to prevent tampering.
- **R21 — Implement Logging Infrastructure:** Set up centralized logging with structured log formats, retention policies, and alerting on security events.
- **R22 — Add Penetration Testing:** Conduct regular penetration testing by qualified security professionals, at minimum before major releases.
- **R23 — Implement Security Training:** Provide regular security awareness training to all development team members on OWASP Top 10 and secure coding practices.
- **R24 — Add Incident Response Plan:** Develop and document an incident response plan with defined roles, communication channels, and escalation procedures.
- **R25 — Implement Data Masking:** Mask sensitive data in non-production environments (e.g., replace real SSNs with generated values in staging/development databases).
- **R26 — Add Two-Factor Authentication:** Implement optional 2FA/MFA for all user accounts, mandatory for admin roles.
- **R27 — Implement IP Whitelisting:** Add optional IP whitelisting for admin access to restrict management functions to approved networks.
- **R28 — Add Database Query Monitoring:** Implement database query monitoring to detect unusual query patterns that may indicate data exfiltration attempts.

10. Data Privacy & GDPR Considerations

As an HRMS platform processing personal data of employees across potentially multiple jurisdictions, NEXUS must address data privacy regulations including GDPR (EU), CCPA (California), and other applicable privacy laws.

10.1 GDPR Compliance Considerations

- **Lawful Basis for Processing:** HR data processing typically relies on "legitimate interest" (employment relationship) and "legal obligation" (tax, labor law compliance). The platform should document the lawful basis for each data processing activity.
- **Data Minimization:** Collect only the minimum personal data necessary for each HR function. Review all data fields to ensure each serves a documented

purpose.

- **Right to Access (Article 15):** Employees should be able to access all personal data held about them. The Employee self-service portal partially addresses this requirement.
- **Right to Rectification (Article 16):** Employees can update certain personal details through self-service. A formal process for rectification requests should be documented.
- **Right to Erasure (Article 17):** Implement a data deletion workflow that handles erasure requests while maintaining legally required retention of employment records.
- **Data Portability (Article 20):** Provide the ability to export employee personal data in a machine-readable format (JSON, CSV).
- **Consent Management:** Track consent for optional data processing activities (e.g., background verification, photo usage) with timestamp and version of the consent form.
- **Data Processing Records (Article 30):** Maintain records of all data processing activities, including purposes, categories of data, recipients, and retention periods.

10.2 Data Retention & Deletion Policy

Data Category	Retention Period	Legal Basis	Deletion Method
Active Employee Records	Duration of employment + 7 years	Legal obligation (tax/labor)	Automated archival after period
Payroll Records	7 years after tax year	Legal obligation (tax)	Scheduled deletion with audit
Recruitment Data (Hired)	Duration of employment + 7 years	Employment contract	Part of employee record deletion
Recruitment Data (Not Hired)	6 months after decision	Consent / Legitimate interest	Automated deletion after period
Attendance Records	3 years	Legal obligation (labor)	Automated archival and deletion
Performance Records	Duration of employment + 3 years	Legitimate interest	Archival with restricted access
Audit Logs	5 years	Legal obligation (compliance)	Automated deletion after period
Login Activity	1 year	Security / Legitimate interest	Automated deletion after period

10.3 Privacy by Design Principles

- **Proactive not Reactive:** Implement privacy measures before data is collected, not after a breach. This includes encryption, access controls, and minimization by default.
- **Privacy as Default Setting:** Default configurations should maximize privacy. For example, employee profile fields should have minimum visibility by default.
- **Privacy Embedded into Design:** Privacy considerations should be part of the development process for every new feature, not an afterthought.
- **Full Functionality — Positive-Sum:** Design features that achieve both functionality and privacy without unnecessary trade-offs.
- **End-to-End Security — Lifecycle Protection:** Protect data from collection through processing to deletion with appropriate security measures at each stage.
- **Visibility and Transparency:** Make data processing practices visible to data subjects through clear privacy notices and accessible data dashboards.
- **Respect for User Privacy:** Prioritize the data subject's interests in all design decisions, providing granular controls and meaningful choices.